

9 GL(M)Ms

9.1 Introduction to GLMs

Generalized linear models (GLMs) extend the linear model (LM) to other (i.e. non-normal) distributions. The idea is the following:

1. We want to keep the regression formula $y \sim f(x)$ of the lm with all its syntax, including splines, random effects and all that. In the GLM world, this is called the “linear predictor”.
2. However, in the GLM, we now allow other residual distributions than normal, e.g. binomial, Poisson, Gamma, etc.
3. Some families require a particular range of the predictions (e.g. binomial requires predictions between zero and one). To achieve this, we use a so-called link function to bring the results of the linear predictor to the desired range.

In R, the distribution is specified via the `family()` function, which distinguishes the glm from the lm function. If you look at the help of family, you will see that the link function is an argument of `family()`. If no link is provided, the default link for the respective family is chosen.

?family

A full GLM structure is thus:

$$y \sim \text{family}[\text{link}^{-1}(f(x) + RE)]$$

Note

As you see, it is noted the link function as link^{-1} in this equation. The reason is that traditionally, the link function is applied to the left hand side of the equation, i.e.

$$\text{link}(y) \sim x$$

This is important to keep in mind when interpreting the names of the link function - the log link, for example, means that $\text{log}(y) = x$, which actually means that we assume $y = \exp(x)$, i.e. the result of the linear predictor enters the exponential function, which assures that we have strictly positive predictions.

The function $f(x)$ itself can have all components that we discussed before, in particular

- You can add random effects as before (using functions `lme4::glmer` or `glmmTMB::glmmTMB`)
- You can also use splines using `mgcv::gam`

9.2 Important GLM variants

The most important are

- Bernoulli / Binomial family = logistic regression with logit link
- Poisson family = Poisson regression with log link
- Beta regression for continuous 0/1 data

Of course, there are many additional distributions that you could consider for your response. Here an overview of the common choices:

Common distributions with typical uses and canonical link functions

Distribution	Support of distribution	Typical uses	Link name	Link function, $\mathbf{X}\beta = g(\mu)$
Normal	real: $(-\infty, +\infty)$	Linear-response data	Identity	$\mathbf{X}\beta = \mu$
Exponential	real: $(0, +\infty)$	Exponential-response data, scale parameters	Negative inverse	$\mathbf{X}\beta = -\mu^{-1}$
Gamma				
Inverse Gaussian	real: $(0, +\infty)$		Inverse squared	$\mathbf{X}\beta = \mu^{-2}$
Poisson	integer: $0, 1, 2, \dots$	count of occurrences in fixed amount of time/space	Log	$\mathbf{X}\beta = \ln(\mu)$
Bernoulli	integer: $\{0, 1\}$	outcome of single yes/no occurrence	Logit	$\mathbf{X}\beta = \ln\left(\frac{\mu}{1 - \mu}\right)$
Binomial	integer: $0, 1, \dots, N$	count of # of "yes" occurrences out of N yes/no occurrences		$\mathbf{X}\beta = \ln\left(\frac{\mu}{n - \mu}\right)$
Categorical	integer: $[0, K)$ K-vector of integer: $[0, 1]$, where exactly one element in the vector has the value 1	outcome of single K-way occurrence		$\mathbf{X}\beta = \ln\left(\frac{\mu}{1 - \mu}\right)$

Multinomial	K -vector of integer: $[0, N]$	count of occurrences of different types (1 .. K) out of N total K -way occurrences	
-------------	----------------------------------	---	--

Screenshot taken from Wikipedia: https://en.wikipedia.org/wiki/Generalized_linear_model#Link_function. Content licensed under the [Creative Commons Attribution-ShareAlike License 3.0](#).

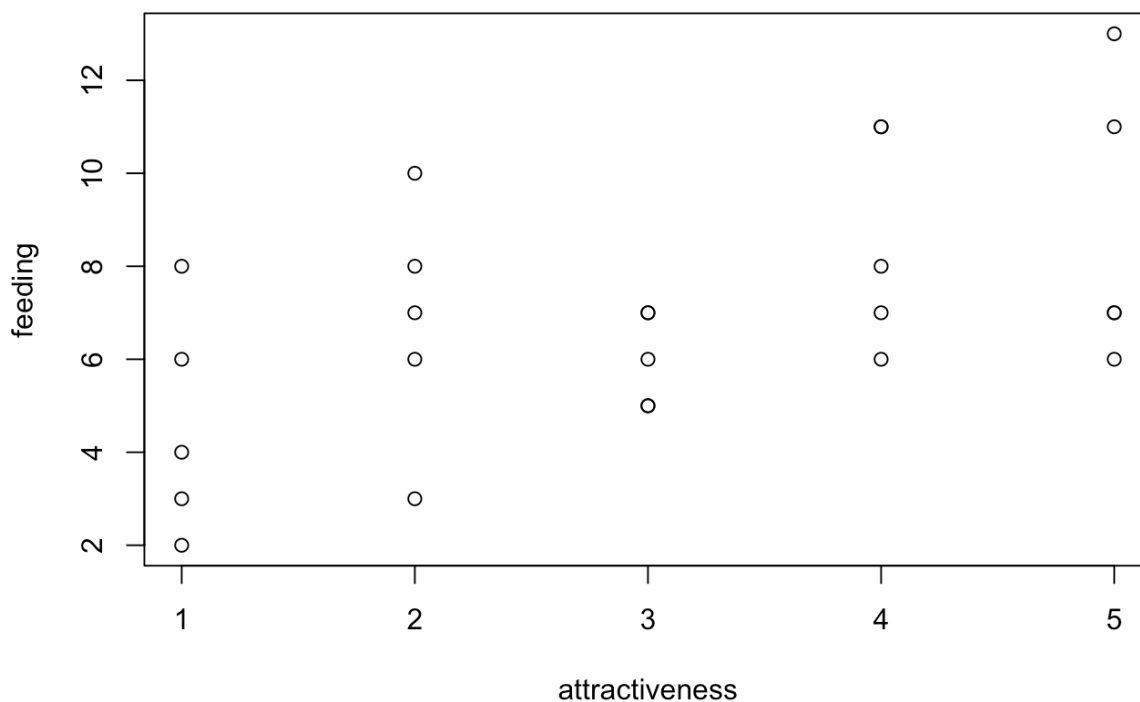
9.2.1 Count data - Poisson regression

The standard model for count data (1,2,3) is the Poisson regression, which implements

- A log-link function $\rightarrow y = \exp(ax + b)$
- A Poisson distribution

As an example, we use the data set birdfeeding from the EcoData package - the dataset consists of observations of foods given to nestlings to parents as a function of the attractiveness of the nestling.

```
library(EcoData)
#str(birdfeeding)
plot(feeding ~ attractiveness, data = birdfeeding)
```



To fit a Poisson regression to this data, we use the `glm()` function, specifying `family = "poisson"`. Note again that the log link function is the default (see `?family`), so it does not have to be specified.

```
fit = glm(feeding ~ attractiveness, data = birdfeeding, family = "poisson")
summary(fit)
```

Call:

```
glm(formula = feeding ~ attractiveness, family = "poisson", data = birdfeeding)
```

Deviance Residuals:

Min	1Q	Median	3Q	Max
-1.55377	-0.72834	0.03699	0.59093	1.54584

Coefficients:

	Estimate	Std. Error	z value	Pr(> z)
(Intercept)	1.47459	0.19443	7.584	3.34e-14 ***
attractiveness	0.14794	0.05437	2.721	0.00651 **

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for poisson family taken to be 1)

Null deviance: 25.829 on 24 degrees of freedom
 Residual deviance: 18.320 on 23 degrees of freedom
 AIC: 115.42

Number of Fisher Scoring iterations: 4

The output is very similar to the `lm()`, however, as the residuals are not any more assumed to scatter normally, all statistics based on R^2 have been replaced by the deviance (deviance = $-2 \log L(\text{saturated}) - \log L(\text{fitted})$). So, we have

- Deviance residuals on top
- Instead of R^2 , we get null vs. residual deviance, and AIC. Based on the deviance, we can calculate a pseudo R^2 , e.g. McFadden, which is $1 - [\log L(M) / \log L(M_0)]$

Note

As already mentioned, the deviance is a generalization of the sum of squares and plays, for example, a similar role as the residual sum of squares in ANOVA.

The deviance of our model M_1 is defined as $Deviance_{M_1} = 2 \cdot (\log L(M_{saturated}) - \log L(M_1))$

Example:

Deviance of M_1

```
fit1 = glm(feeding ~ attractiveness, data = birdfeeding, family = "poisson")
summary(fit1)$deviance
```

```
[1] 18.32001
```

```
fitSat = glm(feeding ~ as.factor(1:nrow(birdfeeding)), data = birdfeeding, family = "poisson")
2*(logLik(fitSat) - logLik(fit1))
```

```
'log Lik.' 18.32001 (df=25)
```

Deviance of the null model (null model is for example needed to calculate the pseudo- R^2):

```
summary(fit1)$null.deviance
```

```
[1] 25.82928
```

```
fitNull = glm(feeding ~ 1, data = birdfeeding, family = "poisson")
2*(logLik(fitSat) - logLik(fitNull))
```

```
'log Lik.' 25.82928 (df=25)
```

Calculation of Pseudo- R^2 with deviance or logL

Common pseudo- R^2 such as McFadden or Nagelkerke use the the logL instead of the deviance, whereas the pseudo- R^2 by Cohen uses the deviance. However, R^2 calculated based on deviance or logL differs considerably, as the former is calculated in reference to the maximal achievable fit (saturated model) while the latter just compares the improvement of fit compared to the null model:

```
# Based on logL
McFadden = 1 - (logLik(fit1)) / (logLik(fitNull))
print(McFadden)
```

```
'log Lik.' 0.06314239 (df=2)
```

```
# Based on deviance
Cohen = 1 - (logLik(fitSat) - logLik(fit1)) / (logLik(fitSat) - logLik(fitNull))
print(Cohen)
```

```
'log Lik.' 0.2907272 (df=25)
```

Note: Unfortunately, there is some confusion about the exact definition, as deviance is sometimes defined (especially outside of the context of GL(M)Ms) simply as $Deviance = -2\text{Log}L(M_1)$

If we want to calculate model predictions, we have to transform to the response scale. Here we have a log link, i.e. we have to transform with $\exp(\text{linear response})$.

```
exp(1.47459 + 3 * 0.14794)
```

```
[1] 6.810122
```

Alternatively (and preferably), you can use the `predict()` function with `type = "response"`

```
dat = data.frame(attractiveness = 3)
predict(fit, newdata = dat) # linear predictor
```

```
1
1.918397
```

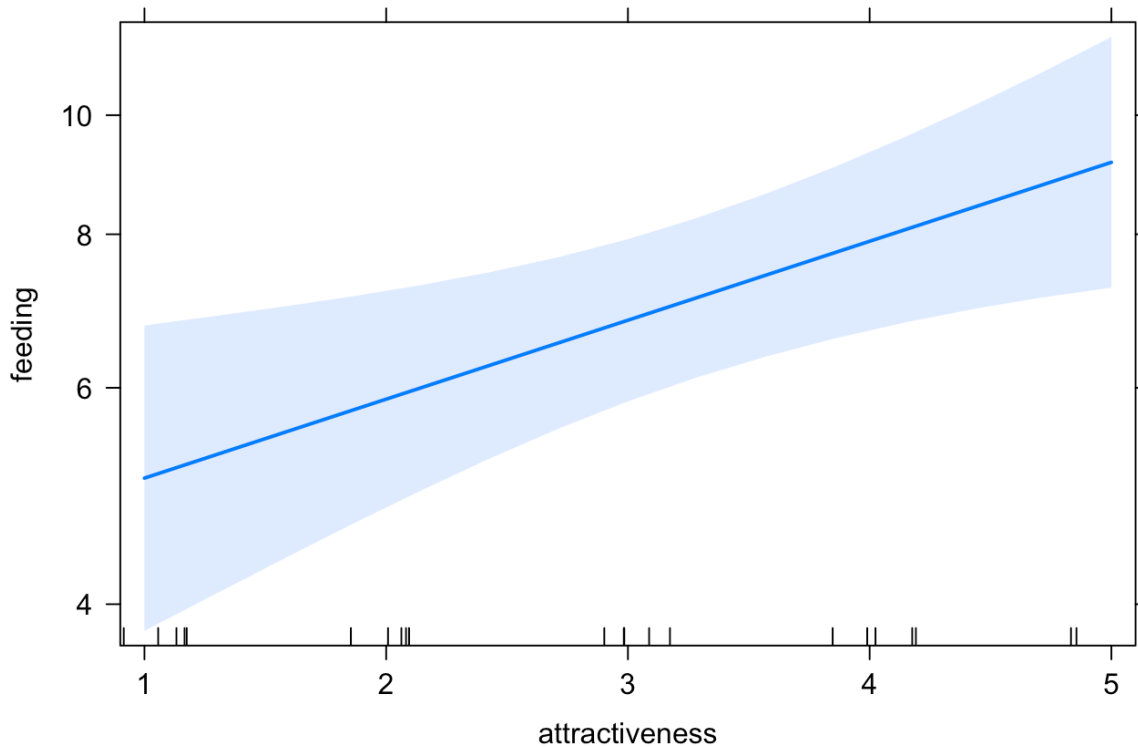
```
predict(fit, newdata = dat, type = "response") # response scale
```

```
1
6.810034
```

Effect plots work as before. Note that the effects package always transforms the y axis according to the link, so we have log scaling on the y axis, and the effect lines remain straight

```
library(effects)
plot(allEffects(fit))
```

attractiveness effect plot



9.2.1.1 Notes on the Poisson regression

Poisson vs. log transformed count data: For count data, even if the distribution is switched, the log link is nearly always the appropriate link function. Before GLMs were widely available, was common to fit lm with log transformed counts, which is basically log link + normal distribution

Log offset: If there is a variable that controls the number of counts (e.g. time, area), this variable is usually added as an offset in the following form

```
fit = glm(y ~ x + offset(log(area)), family = "poisson")
```

As the log-link connects the linear predictor as in $y = \exp(x)$, and $\exp(x + \log(\text{area})) = \exp(x) \cdot \text{area}$, this makes the expected counts proportional to area, or whatever variable is added as a log offset.

Interactions: As for all GLMs with nonlinear link functions, interpretation of the interactions is more complicated. See notes on this below.

9.2.2 0/1 or k/n data - logistic regression

The standard model to fit binomial (0/1 or k/n) data is the logistic regression, which combines the binomial distribution with a logit link function. To get to know this model, let's have a look at the titanic data set in `EcoData`:

```
library(EcoData)
#str(titanic)
#mosaicplot( ~ survived + sex + pclass, data = titanic)
titanic$pclass = as.factor(titanic$pclass)
```

We want to analyze how survival in the titanic accident depended on other predictors. We could fit an lm, but the residual checks make it very evident that the data with a 0/1 response don't fit to the assumption of an lm:

```
fit = lm(survived ~ sex * age, data = titanic)
summary(fit)
```

Call:

```
lm(formula = survived ~ sex * age, data = titanic)
```

Residuals:

	Min	1Q	Median	3Q	Max
	-0.8901	-0.2291	-0.1564	0.2612	0.9744

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	0.637645	0.046165	13.812	< 2e-16 ***
sexmale	-0.321308	0.059757	-5.377	9.35e-08 ***
age	0.004006	0.001435	2.792	0.00534 **
sexmale:age	-0.007641	0.001823	-4.192	3.01e-05 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

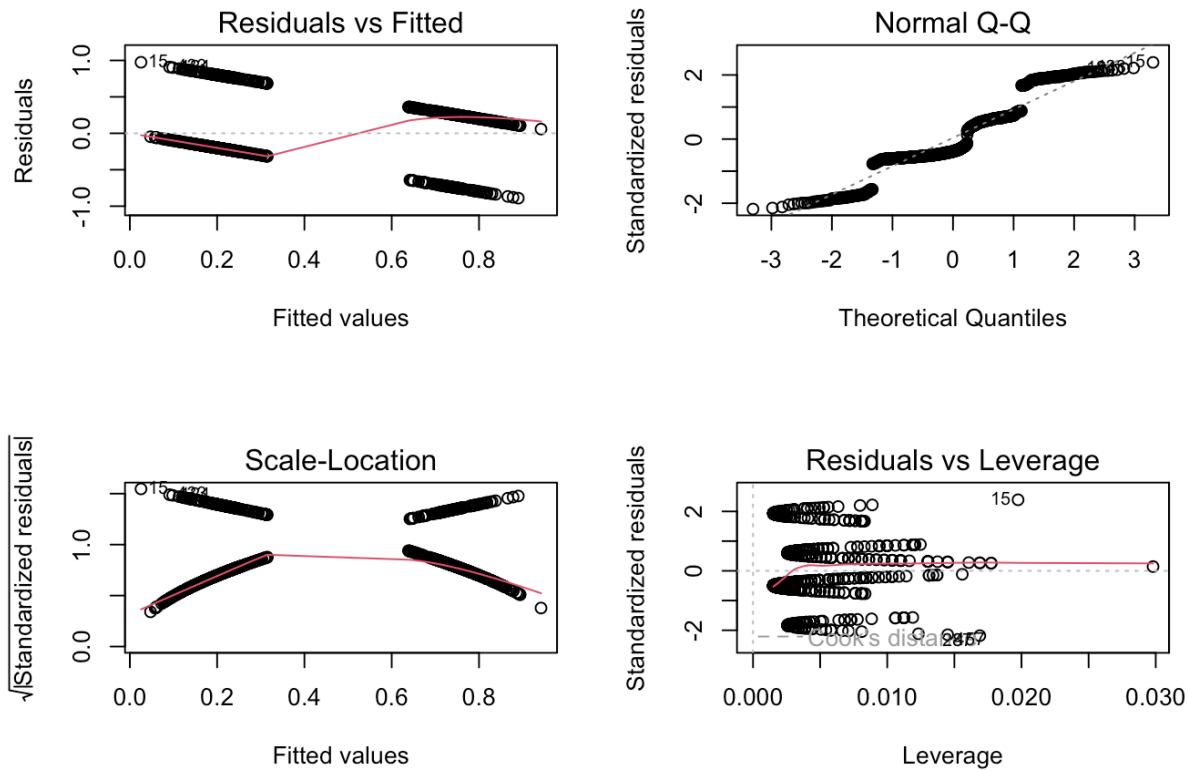
Residual standard error: 0.4115 on 1042 degrees of freedom

(263 observations deleted due to missingness)

Multiple R-squared: 0.3017, Adjusted R-squared: 0.2997

F-statistic: 150 on 3 and 1042 DF, p-value: < 2.2e-16

```
par(mfrow = c(2, 2))
plot(fit)
```

Thus, let's move to the logistic regression, which assumes a 0/1 response + logit link. In principle, this distribution is called Bernoulli, but in R both 0/1 and k/n are called "binomial", as Bernoulli is the special case of binomial where $n = 1$.

```
m1 = glm(survived ~ sex*age, family = "binomial", data = titanic)
summary(m1)
```

Call:

```
glm(formula = survived ~ sex * age, family = "binomial", data = titanic)
```

Deviance Residuals:

Min	1Q	Median	3Q	Max
-2.0247	-0.7158	-0.5776	0.7707	2.2960

Coefficients:

	Estimate	Std. Error	z value	Pr(> z)
(Intercept)	0.493381	0.254188	1.941	0.052257 .
sexmale	-1.154139	0.339337	-3.401	0.000671 ***
age	0.022516	0.008535	2.638	0.008342 **
sexmale:age	-0.046276	0.011216	-4.126	3.69e-05 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for binomial family taken to be 1)

Null deviance: 1414.6 on 1045 degrees of freedom
 Residual deviance: 1083.4 on 1042 degrees of freedom
 (263 observations deleted due to missingness)
 AIC: 1091.4

Number of Fisher Scoring iterations: 4

Note

The syntax here is for 0/1 data. If you have k/n data, you can either specify the response as `cbind(k, n-k)`, or you can fit the glm with `k ~ x, weights = n`

The interpretation of the regression table remains unchanged. To transform to predictions, we have to use the inverse logit, which is in R:

```
plogis(0.493381 + 0.022516 * 20) # Women, age 20.
```

```
[1] 0.7198466
```

```
plogis(0.493381 - 1.154139 + 20*(0.022516 - 0.046276)) # Men, age 20
```

```
[1] 0.2430632
```

Alternatively, we can again use the predict function

```
newDat = data.frame(sex = as.factor(c("female", "male")), age = c(20, 20))
predict(m1, newdata = newDat) # Linear predictor.
```

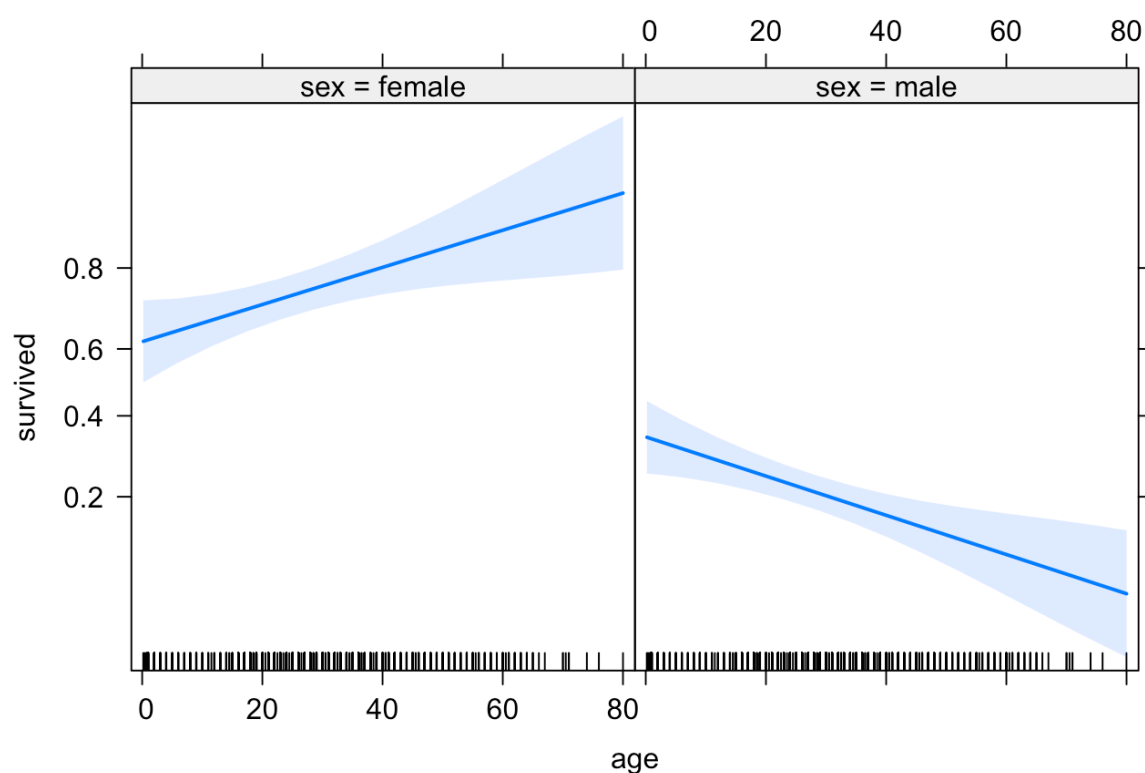
```
      1      2
0.9436919 -1.1359580
```

```
predict(m1, newdata = newDat, type = "response") # Response scale.
```

```
      1      2
0.7198448 0.2430633
```

Finally, the effect plots - note again the scaling of the y axis, which is now logit.

```
library(effects)
plot(allEffects(m1))
```

sex*age effect plot

If you do an ANOVA on a glm, you should take care that you perform a Chisq and not an F-test. You notice that you have the right one if the ANOVA doesn't supply SumSq statistics, but deviance. In the anova function, we have to set this by hand

```
anova(m1, test = "Chisq")
```

Analysis of Deviance Table

Model: binomial, link: logit

Response: survived

Terms added sequentially (first to last)

	Df	Deviance	Resid. Df	Resid. Dev	Pr(>Chi)
NULL			1045	1414.6	
sex	1	312.612	1044	1102.0	< 2.2e-16 ***
age	1	0.669	1043	1101.3	0.4133
sex:age	1	17.903	1042	1083.4	2.324e-05 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

note that anova works for glm and glmmTMB, but not for lme4::glmer.

The function car::Anova works for all models and uses a ChiSq test automatically. Given that you probably want to use a type II or III Anova anyway, you should prefer it.

```
car::Anova(m1)
```

Analysis of Deviance Table (Type II tests)

Response: survived

	LR	Chisq	Df	Pr(>Chisq)
sex	310.044	1	< 2.2e-16	***
age	0.669	1	0.4133	
sex:age	17.903	1	2.324e-05	***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Of course, when using this with random effects, the caveats that we discussed when introducing random effects apply: this function does not account for changes in the degrees of freedom created by changing the fixed effect structure. Something like the lmerTest package which uses a df approximation does not exist for GLMMs. Thus, the values that you obtain here are calculated under the assumption that the RE structure is the same.

If you see large changes in your RE structure, or if you want to select on the REs, you can move to a simulated LRT.

9.2.2.1 Notes on the logistic regression

Offset: there is no exact solution for making 0/1 data dependent on a scaling factor via an offset, which is often desirable, for example in the context of survival analysis with different exposure times. An approximate solution is to use an offset together with the log-log link (instead of logit).

Interactions: As for all GLMs with nonlinear link functions, interpretation of the interactions is more complicated. See notes in this below.

Overdispersion: 0/1 poisson responses cannot be overdispersed, but k/n responses can be. However, 0/1 responses can show overdispersion if grouped to k/n. Note next section on residual checks, as well as comments on testing binomial GLMs in the [DHARMA vignette](#).

Example - Elk Data

You will be given a data set of habitat use of Elks in Canada. Measured is the presence of Elks (0/1), and a number of other predictors. Description of variables:

- dist_roads - distance of the location to the next road
- NDVI - normalized difference vegetation index, essentially greenness of vegetation on the site
- ruggedness of the terrain
- dem - digital elevation model = elevation above sea level
- presence - presence of the elk
- habitat - open or forest

Perform either:

- A predictive analysis, i.e. a model to predict where Elks can be found.
- A causal analysis, trying to understand the effect of roads on Elk presence.

Solution

9.2.3 Continous positive response - Gamma regression

```
library(faraway)
```

Attaching package: 'faraway'

The following objects are masked from 'package:EcoData':

melanoma, rats

```
fit <- lm(log(resist) ~ x1 + x2 + x3 + x4, data = wafer)
summary(fit)
```

Call:

```
lm(formula = log(resist) ~ x1 + x2 + x3 + x4, data = wafer)
```

Residuals:

	Min	1Q	Median	3Q	Max
	-0.17572	-0.06222	0.01749	0.08765	0.10841

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	5.44048	0.05982	90.948	< 2e-16 ***
x1+	0.12277	0.05350	2.295	0.042432 *
x2+	-0.29986	0.05350	-5.604	0.000159 ***
x3+	0.17844	0.05350	3.335	0.006652 **
x4+	-0.05615	0.05350	-1.049	0.316515

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.107 on 11 degrees of freedom

Multiple R-squared: 0.8164, Adjusted R-squared: 0.7496

F-statistic: 12.22 on 4 and 11 DF, p-value: 0.0004915

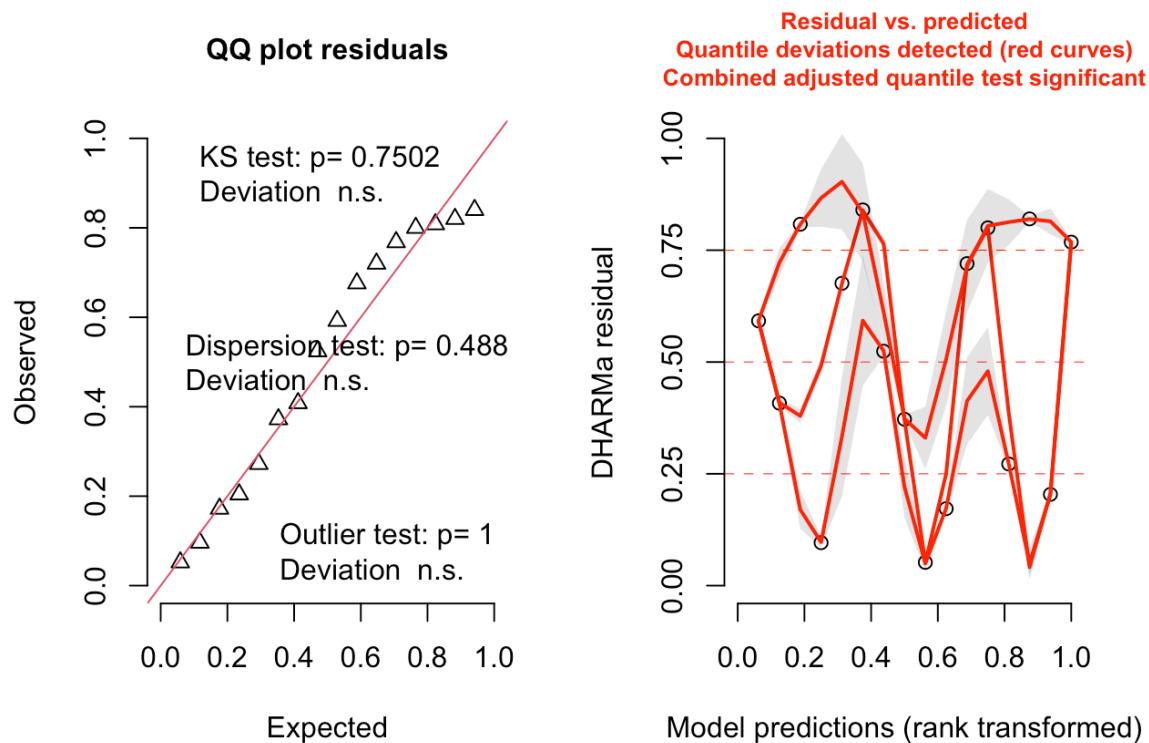
```
library(DHARMA)
simulateResiduals(fit, plot = T)
```

We had to increase `err` for some of the quantiles. See fit\$calibr\$err

We had to increase `err` for some of the quantiles. See fit\$calibr\$err

We had to increase `err` for some of the quantiles. See fit\$calibr\$err

DHARMA residual



Object of Class DHARMA with simulated residuals based on 250 simulations with `refit = FALSE` . See ? DHARMA::simulateResiduals for help.

Scaled residual values: 0.052 0.272 0.408 0.676 0.82 0.768 0.84 0.172 0.524 0.72 0.592 0.808 0.8 0.204 0.096 0.372

Alternative: Gamma regression

```
fit <- glm(formula = resist ~ x1 + x2 + x3 + x4,
           family = Gamma(link = "log"),
           data = wafer)
summary(fit)
```

Call:

```
glm(formula = resist ~ x1 + x2 + x3 + x4, family = Gamma(link = "log"),
    data = wafer)
```

Deviance Residuals:

Min	1Q	Median	3Q	Max
-0.17548	-0.06486	0.01423	0.08399	0.10898

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	5.44552	0.05856	92.983	< 2e-16 ***
x1+	0.12115	0.05238	2.313	0.041090 *
x2+	-0.30049	0.05238	-5.736	0.000131 ***
x3+	0.17979	0.05238	3.432	0.005601 **
x4+	-0.05757	0.05238	-1.099	0.295248

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for Gamma family taken to be 0.01097542)

Null deviance: 0.69784 on 15 degrees of freedom
Residual deviance: 0.12418 on 11 degrees of freedom
AIC: 152.91

Number of Fisher Scoring iterations: 4

Note

For Gamma regression, different link functions are used. The canonical link, which is also the default in R, is the inverse link $1/x$. However, also log and identity link are commonly used.

9.2.4 Continuous proportions - Beta regression and other options

We already covered discrete proportions, which can be modelled with a logistic regression. For continuous proportions, this model is not suitable, because we don't know how many "trials" we have. There are a few other options to model this data, in particular the beta regression. Let's have a look.

There are two main ways to fit this data:

1. Transform the response, or fit the GLM with a logit link or the arcsine transformation on the response
2. A Beta regression

For further options, see [here](#). I would generally recommend that the Beta regression is currently the preferred way and the first thing I would try. A good way to fit it is the package `glmmTMB`.

Case study

Here a case study, using the `EcoData` dataset

```
?elemental
```

traditional `lm`, arc sine transformed response of proportions, see critique in <https://esajournals.onlinelibrary.wiley.com/doi/10.1890/10-0340.1>

```
m1 <- lm(N_arc ~ Year + Site, data = elemental[elemental$Species == "ABBA", ])
summary(m1)
```

Call:

```
lm(formula = N_arc ~ Year + Site, data = elemental[elemental$Species ==
"ABBA", ])
```

Residuals:

Min	1Q	Median	3Q	Max
-0.016553	-0.006151	-0.001069	0.004322	0.028149

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	-1.719e+01	4.323e+00	-3.975	0.000127 ***
Year	8.570e-03	2.145e-03	3.996	0.000117 ***
SiteDP	3.880e-03	5.972e-03	0.650	0.517275
SiteTN	1.043e-02	6.191e-03	1.685	0.094856 .
SiteUNI	4.646e-04	5.971e-03	0.078	0.938124

```
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
Residual standard error: 0.00826 on 109 degrees of freedom
Multiple R-squared:  0.2968,    Adjusted R-squared:  0.271
F-statistic: 11.5 on 4 and 109 DF,  p-value: 7.936e-08
Now a beta regression
```

```
library(glmmTMB)
m2 <- glmmTMB(N_dec ~ Year + Site, family = beta_family, data = elemental[elemental$Species == "ABBA", ] )
summary(m2)
```

```
Family: beta ( logit )
Formula:      N_dec ~ Year + Site
Data: elemental[elemental$Species == "ABBA", ]
```

AIC	BIC	logLik	deviance	df.resid
-1148.0	-1131.6	580.0	-1160.0	108

```
Dispersion parameter for beta family (): 3.9e+03
```

```
Conditional model:
```

	Estimate	Std. Error	z value	Pr(> z)
(Intercept)	-361.33063	82.66891	-4.371	1.24e-05 ***
Year	0.17684	0.04101	4.313	1.61e-05 ***
SiteDP	0.08905	0.12851	0.693	0.4883
SiteTN	0.22115	0.13190	1.677	0.0936 .
SiteUNI	0.01559	0.12865	0.121	0.9035

```
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

9.3 Residual and their solutions in GLMs

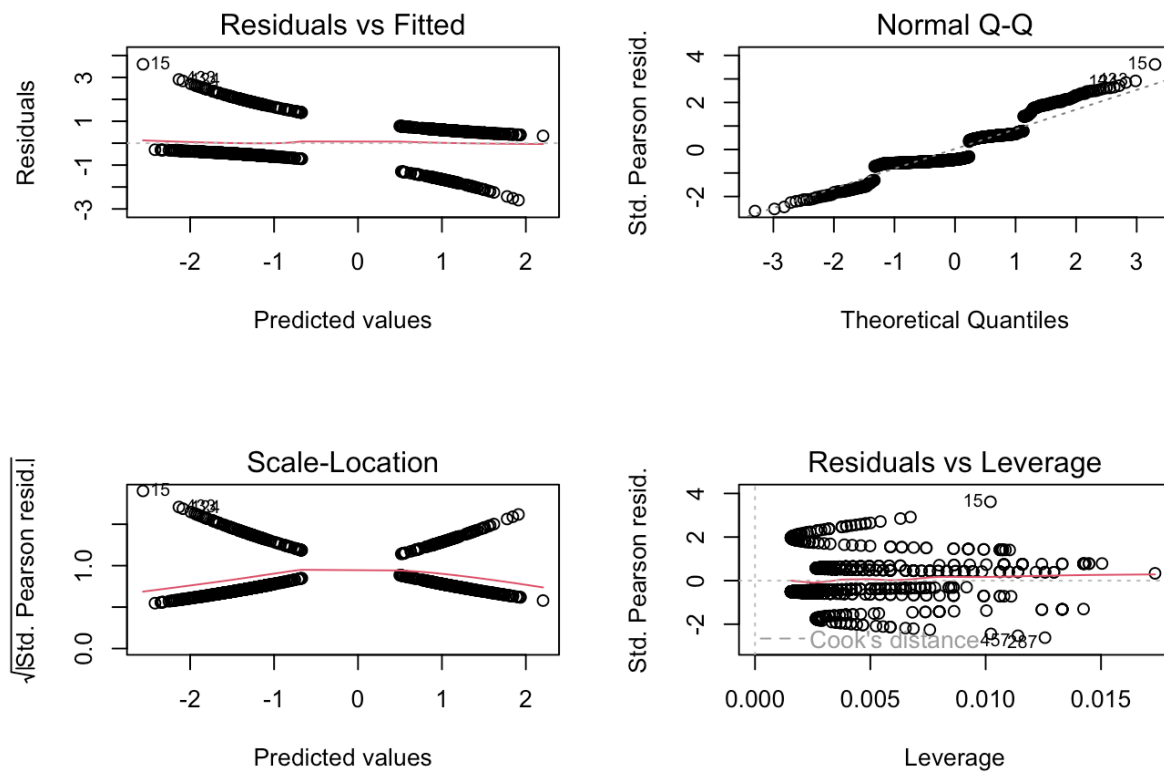
First of all: everything we said about model selection and residual checks for LMs also apply for GLMs, with only very few additions, so you should check your model in principle as before. However, there are a few tweaks you have to be aware of.

Let's look again at the titanic example

```
m1 = glm(survived ~ sex*age, family = "binomial", data = titanic)
```

How can we check the residuals of this model? Due to an unfortunate programming choice in R (Nerds: Check `class(m1)`), the standard residual plots still work

```
par(mfrow = c(2, 2))
plot(m1)
```

but they look horrible, because they still check for normality of the residuals, while we are interested in the question of whether the residuals are binomially distributed.

9.3.1 DHARMA residual plots for GL(M)Ms

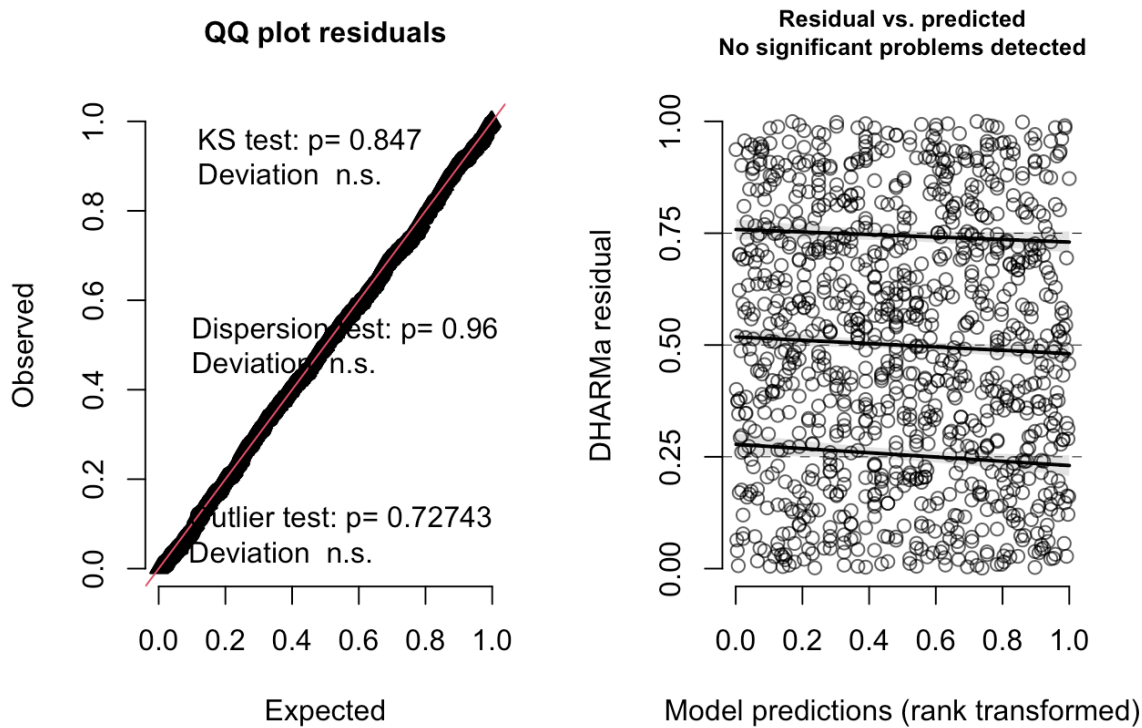
The `DHARMA` package that we already introduced solves this problem

```
library(DHARMA)
res = simulateResiduals(m1)
```

Standard plot:

```
plot(res)
```

DHARMA residual



Out of the help page: The function creates a plot with two panels. The left panel is a uniform Q-Q plot (calling [plotQQunif](#)), and the right panel shows residuals against predicted values (calling [plotResiduals](#)), with outliers highlighted in red.

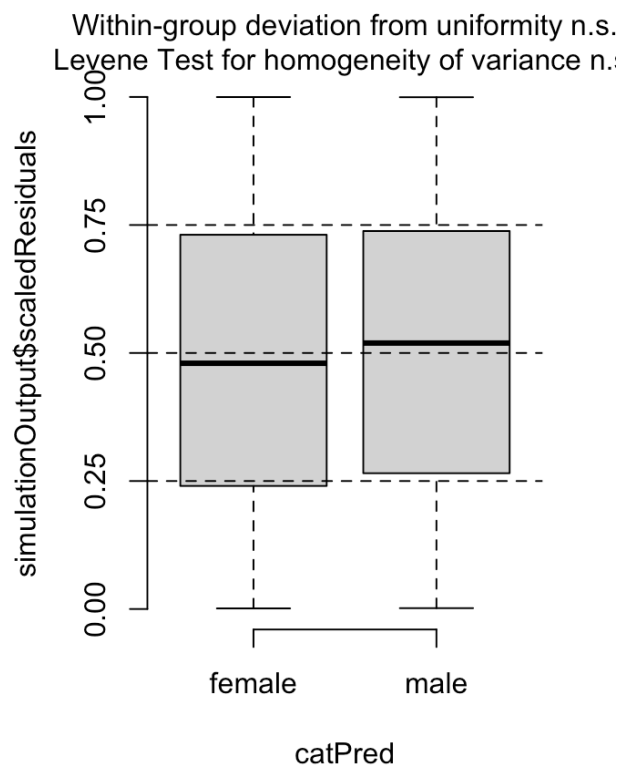
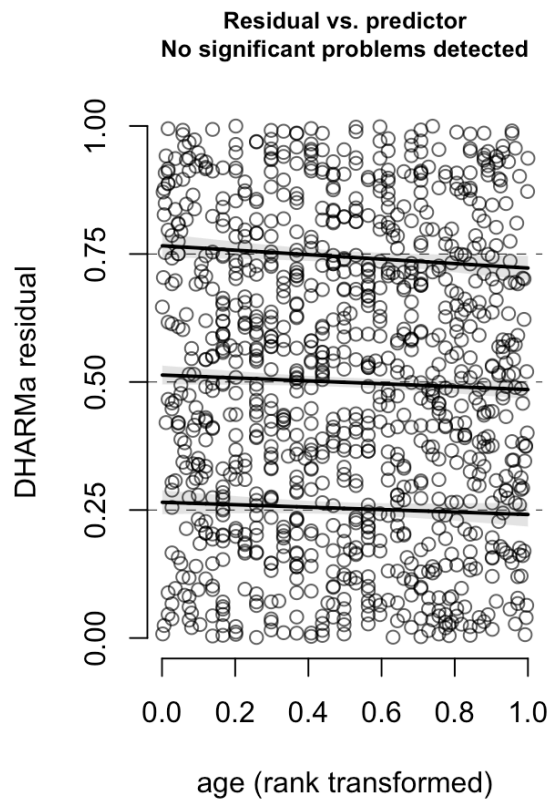
Very briefly, we would expect that a correctly specified model shows:

- A straight 1-1 line, as well as not significant of the displayed tests in the Q-Q-plot (left) -> Evidence for a correct overall residual distribution (for more details on the interpretation of this plot, see help).
- Visual homogeneity of residuals in both vertical and horizontal direction, as well as no significance of quantile tests in the Residual vs. predicted plot (for more details on the interpretation of this plot, see help).

Deviations from these expectations can be interpreted similarly to a linear regression. See the vignette for detailed examples.

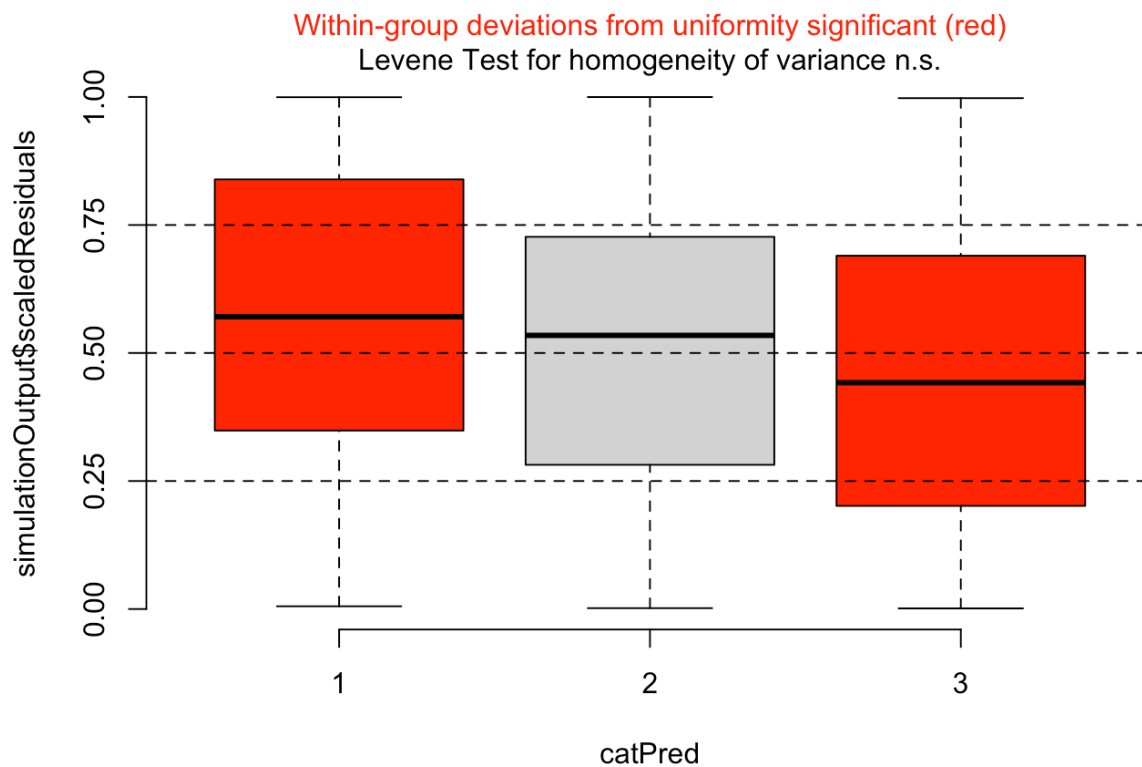
With that in mind, we can say that there is nothing special to see here. Also residuals against predictors shows no particular problem:

```
par(mfrow = c(1, 2))
plotResiduals(m1, form = model.frame(m1)$age)
plotResiduals(m1, form = model.frame(m1)$sex)
```



However, residuals against the missing predictor pclass show a clear problem:

```
dataUsed = as.numeric(rownames(model.frame(m1)))
plotResiduals(m1, form = titanic$pclass[dataUsed])
```



Thus, I should add passenger class to the model

```
m2 = glm(survived ~ sex*age + pclass, family = "binomial", data = titanic)
summary(m2)
```

Call:

```
glm(formula = survived ~ sex * age + pclass, family = "binomial",
    data = titanic)
```

Deviance Residuals:

Min	1Q	Median	3Q	Max
-2.3844	-0.6721	-0.4063	0.7041	2.5440

Coefficients:

	Estimate	Std. Error	z value	Pr(> z)	
(Intercept)	2.790839	0.362822	7.692	1.45e-14	***
sexmale	-1.029755	0.358593	-2.872	0.00408	**
age	-0.004084	0.009461	-0.432	0.66598	
pclass2	-1.424582	0.241513	-5.899	3.67e-09	***
pclass3	-2.388178	0.236380	-10.103	< 2e-16	***
sexmale:age	-0.052891	0.012025	-4.398	1.09e-05	***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for binomial family taken to be 1)

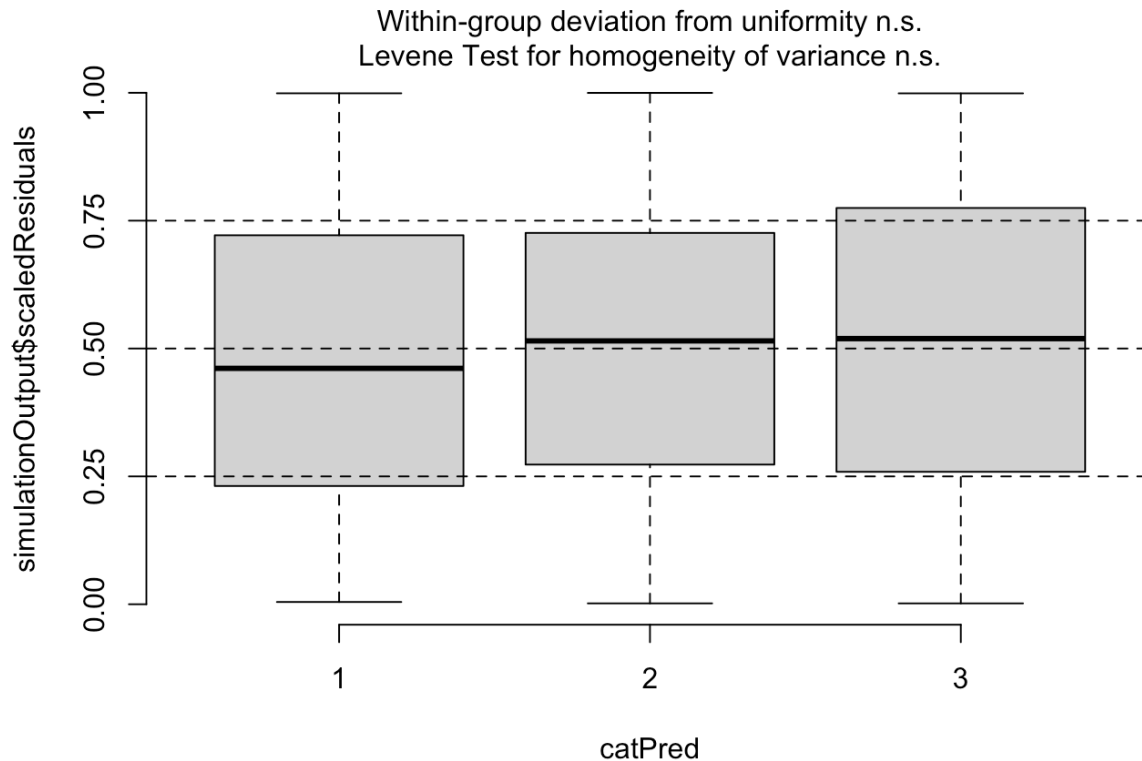
Null deviance: 1414.62 on 1045 degrees of freedom
Residual deviance: 961.92 on 1040 degrees of freedom

(263 observations deleted due to missingness)

AIC: 973.92

Number of Fisher Scoring iterations: 5

```
plotResiduals(m2, form = model.frame(m2)$pclass)
```



Now, residuals look fine. Of course, if your model gets more complicated, you may want to do additional checks, for example for the distribution of random effects etc.

9.3.2 Dispersion Problems in GLMs

One thing that is different between GLMs and LM is that GLMs can display overall dispersion problems. The most common GLMs to show overdispersion are the Poisson and the logistic regression.

The reason is that simple GLM distributions such as the Poisson or the Binomial (for k/n data) do not have a parameter for adjusting the spread of the observed data around the regression line (dispersion), but their variance is a fixed as function of the mean.

There are good reasons for why this is the case (Poisson and Binomial describe particular processes, e.g. coin flip, for which the variance is a fixed function of the mean), but the fact is that when applying these GLMs on real data, we often find overdispersion (more dispersion than expected), and more rarely, underdispersion (less dispersion than expected).

To remove the assumptions of a fixed dispersion, there are three options, of which you should definitely take the third one:

1. **Quasi-distributions**, which are available in `glm`. Those add a term to the likelihood that corrects the p-values for the dispersion, but they are not distributions. -> Can't check residuals, no AIC. -> Discouraged.

2. **Observation-level random effect (OLRE)** - Add a separate random effect per observation. This effectively creates a normal random variate at the level of the linear predictor, increases variance on the responses.
3. A **GLM distribution with variable dispersion**, for Poisson usually the negative binomial.

The reason why we should prefer the 3rd option is that it allows better residual checks and to model the dispersion as a function of the predictors, see next section.

Note

Overdispersion is often created by model misfit. Thus, before moving to a variable dispersion GLM, you should check for / correct model misfit.

9.3.2.1 Recognizing overdispersion

To understand how to recognize overdispersion, let's look at an example. We'll use the Salamanders dataset from the package glmmTMB, starting with a simple Poisson glm:

```
library(glmmTMB)
library(lme4)
```

Loading required package: Matrix

Attaching package: 'lme4'

The following object is masked from 'package:nlme':

lmList

```
library(DHARMA)

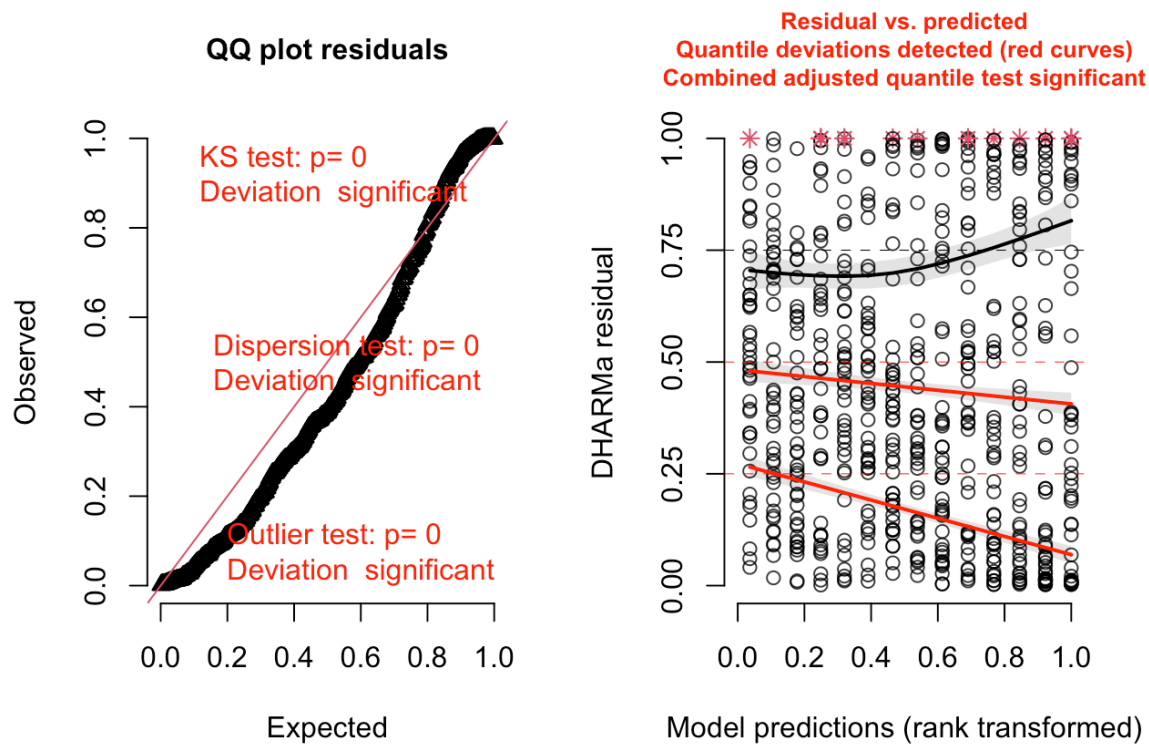
m1 = glm(count ~ spp + mined, family = poisson, data = Salamanders)
```

Overdispersion will be automatically highlighted in the standard DHARMA plots

```
res = simulateResiduals(m1, plot = T)
```

DHARMA:testOutliers with type = binomial may have inflated Type I error rates for integer-valued distributions. To get a more exact result, it is recommended to re-run testOutliers with type = 'bootstrap'. See ?testOutliers for details

DHARMA residual



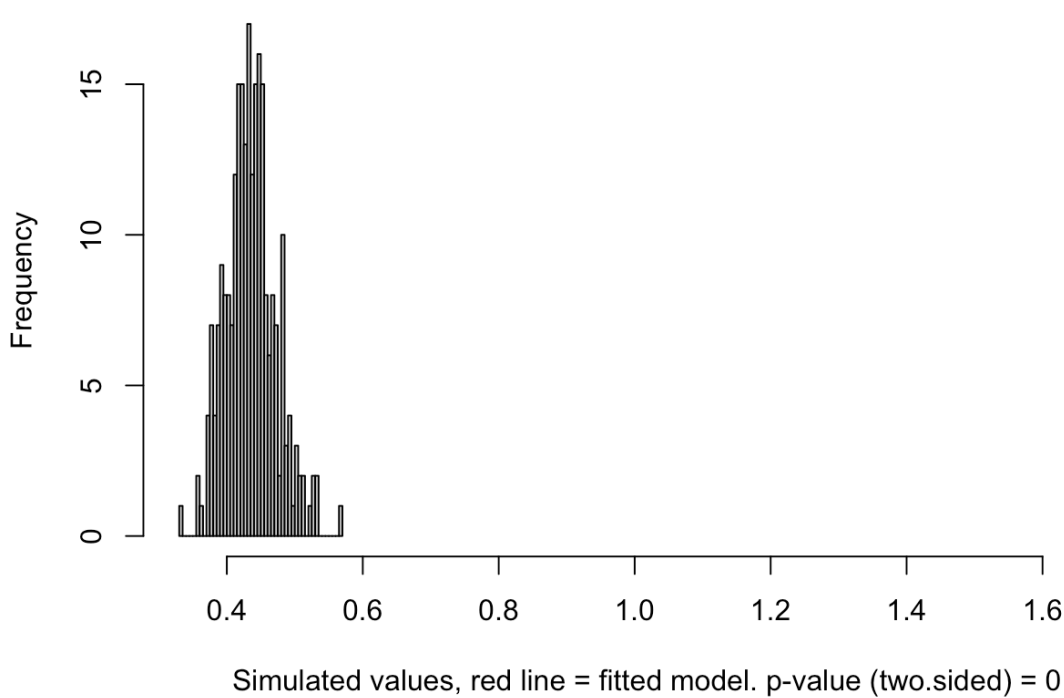
You see the dispersion problem by:

- Dispersion test in the left plot significant
- QQ plot S-shaped
- Quantile lines in the right plots outside their expected quantiles

You can get a more detailed output with the `testDispersion` function, which also displays the direction of the dispersion problem (over or underdispersion)

```
testDispersion(res)
```

DHARMA nonparametric dispersion test via sd of residuals fitted vs. simulated



DHARMA nonparametric dispersion test via sd of residuals fitted vs. simulated

```
data: simulationOutput
dispersion = 3.9152, p-value < 2.2e-16
alternative hypothesis: two.sided
```

OK, often the dispersion problem is caused by structural problems. Let's add a random effect for site, which makes sense. We can do so using the function `lme4::glmer`, which adds the same extensions to `glm` as `lmer` adds to `lm`. This means we can use the usual random effect syntax we have used before.

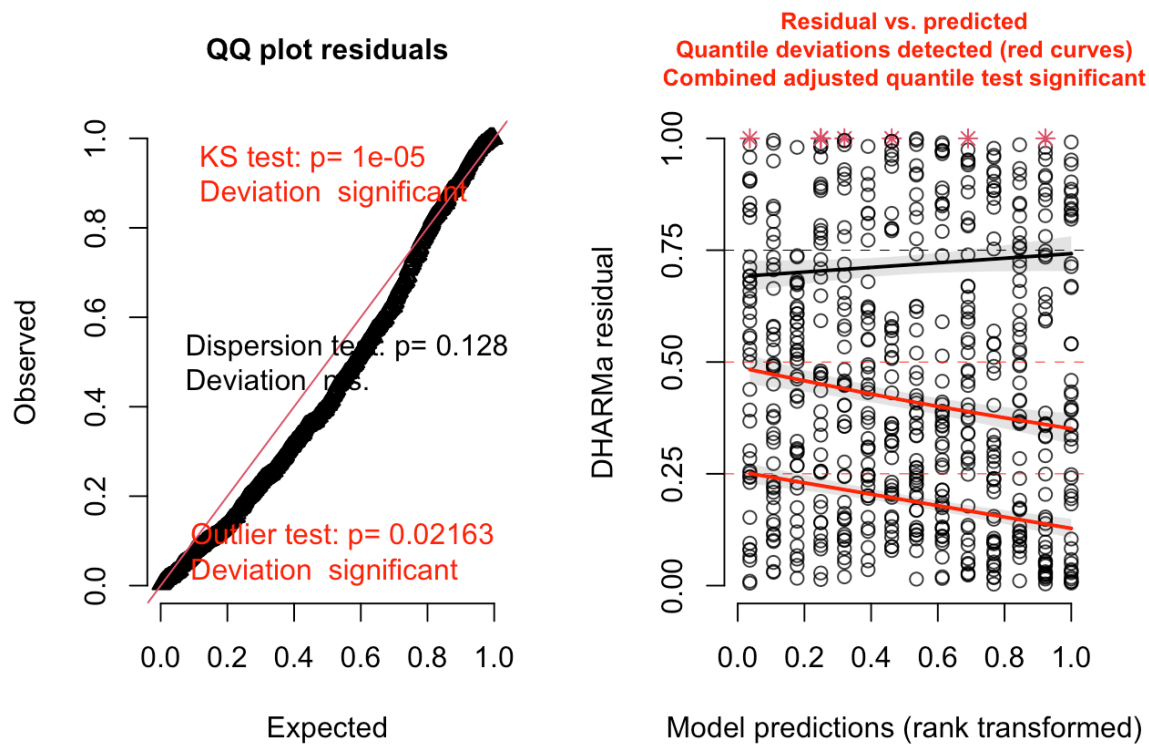
```
m2 = glmer(count ~ spp + mined + (1|site),
            family = poisson, data = Salamanders)
```

The standard dispersion test is OK

```
res = simulateResiduals(m2, plot = T)
```

`DHARMA::testOutliers` with `type = binomial` may have inflated Type I error rates for integer-valued distributions. To get a more exact result, it is recommended to re-run `testOutliers` with `type = 'bootstrap'`. See `?testOutliers` for details

DHARMA residual

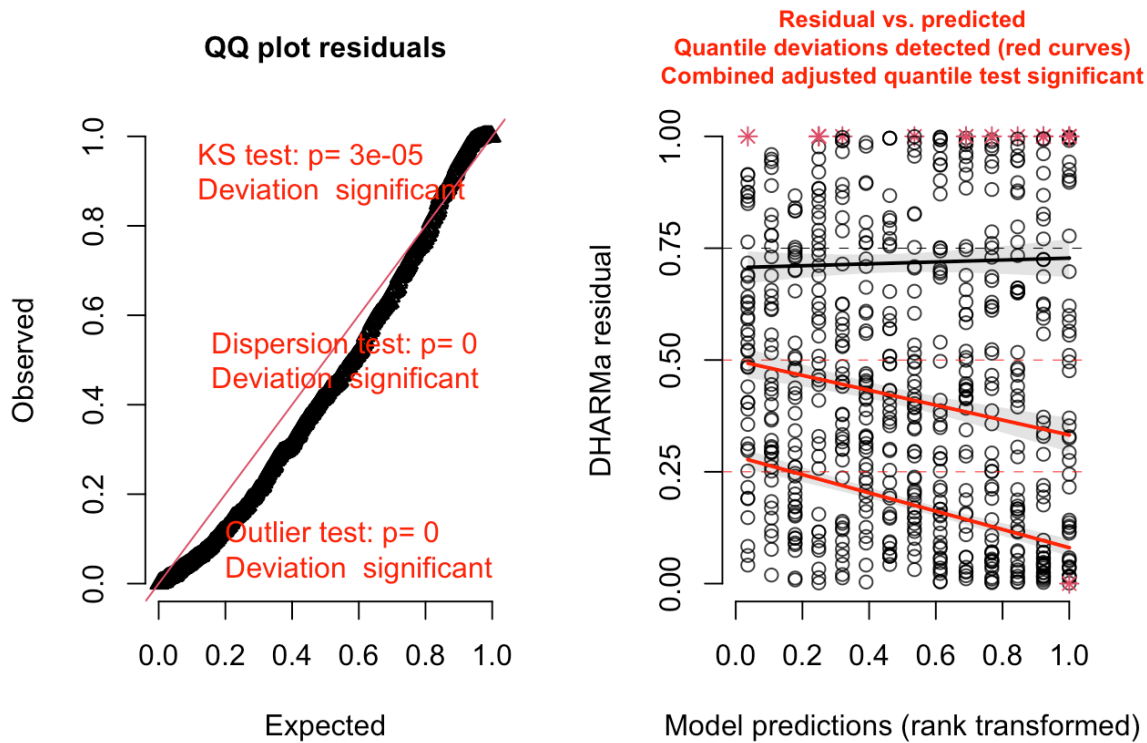


But when random effects are added, you should prefer to calculate conditional residuals, because this test is more powerful. For lme4 models, we can switch via `re.form = T`

```
res = simulateResiduals(m2, plot = T, re.form = NULL)
```

DHARMA:testOutliers with `type = binomial` may have inflated Type I error rates for integer-valued distributions. To get a more exact result, it is recommended to re-run `testOutliers` with `type = 'bootstrap'`. See `?testOutliers` for details

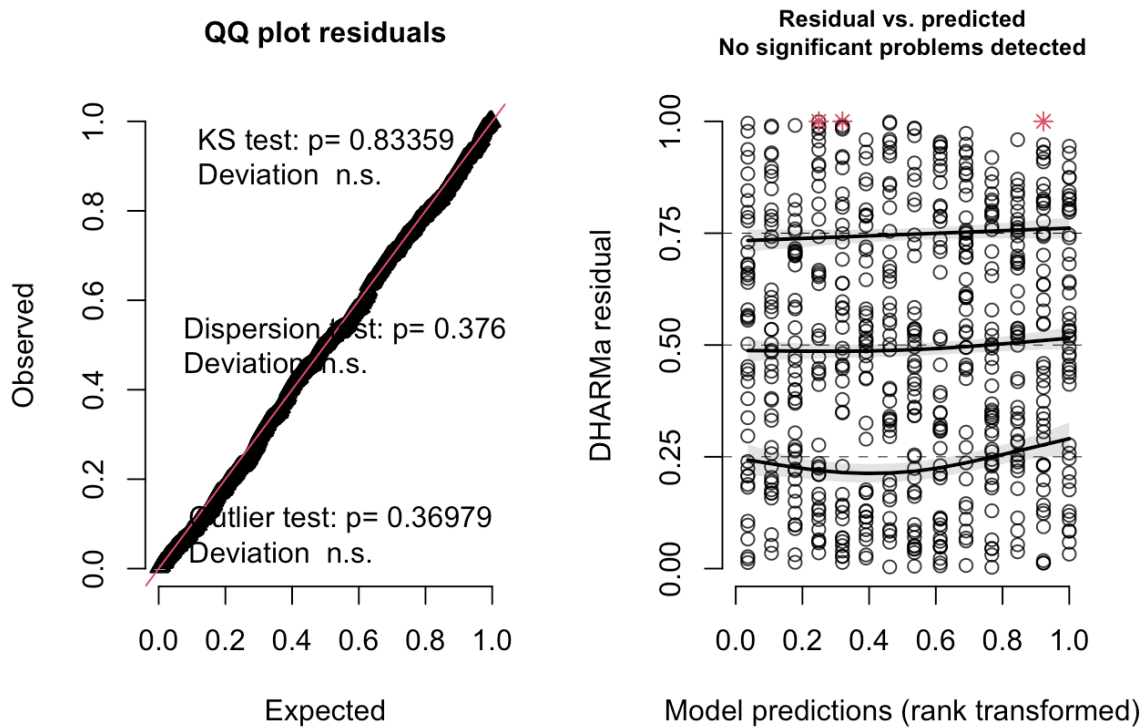
DHARMA residual



This test shows that there is still some overdispersion. Actually, what the plots also show is heteroskedasticity, and we should probably deal with that as well, but we will only learn how in the next chapter. For now, let's switch to a negative binomial model. This could be fit with `lme4`, but it is more convenient to use the package `glmmTMB`, which has the same syntax as `lme4`, but more advanced options.

```
m4 = glmmTMB(count ~ spp + mined + (1|site), family = nbinom2, data = Salamanders)
res = simulateResiduals(m4, plot = T)
```

DHARMA residual



Unfortunately, glmmTMB doesn't allow to calculate conditional residuals, so we have to be satisfied with the fact that the unconditional residuals look great.

9.3.3 Zero-inflation

Another common problem in count data (Poisson / negative binomial), but also other GLMs (e.g. binomial, beta) is that the observed data has more zeros than expected by the fitted distribution. For the beta, 1-inflation, and for the k/n binomial, n-inflation is also common, and tested for / addressed in the same way.

To deal with this **zero-inflation**, one usually adds an additional model component that controls how many zeros are produced. The default way to do this is assuming two separate processes which act after one another:

1. First, we have the normal GLM, predicting what values we would expect
2. On top of that, we have a logistic regression, which decides whether the GLM prediction or a zero should be observed

Note that the result of 1 can also be zero, so there are two explanations for a zero in the data. Zero-inflated GLMMs can, for example, be fit with glmmTMB, using the `ziformula` argument.

9.3.3.1 Recognizing zero-inflation

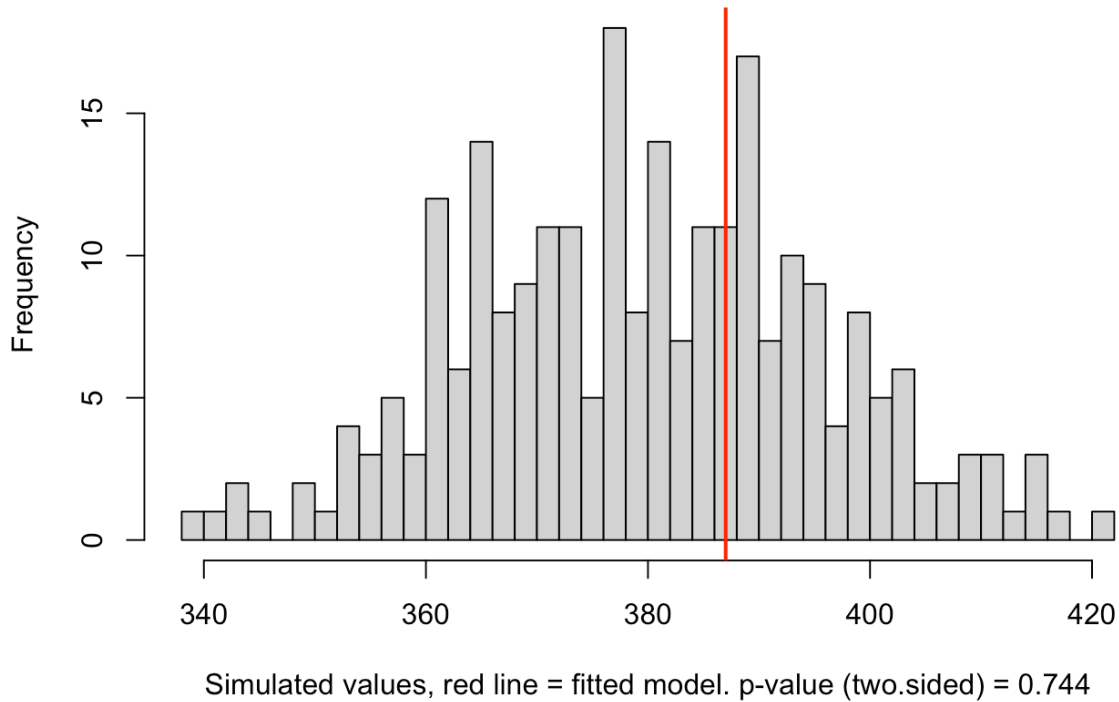
Caution

The fact that you have a lot of zeros in your data does not indicate zero-inflation. Zero-inflation is with respect to the fitted model. You can only check for zero-inflation after fitting a model.

Let's look at our last model - DHARMA has a special function to check for zero-inflation

```
testZeroInflation(res)
```

**DHARMA zero-inflation test via comparison to
expected zeros with simulation under H_0 = fitted
model**



DHARMA zero-inflation test via comparison to expected zeros with
simulation under H_0 = fitted model

```
data: simulationOutput
ratioObsSim = 1.0172, p-value = 0.744
alternative hypothesis: two.sided
```

This shows no sign of zero-inflation. There are, however, two problems with this test:

1. glmmTMB models only allow unconditional residuals, which means that dispersion and zero-inflation tests are less powerfull
2. When there is really zero-inflation, variable dispersion models such as the negative Binomial often simply increase the dispersion to account for the zeros, leading to no apparent zero-inflation in the residuals, but rather underdispersion.

Thus, for zero-inflation, model selection, or simply fitting a ZIP model is often more reliable than residual checks. You can compare a zero-inflation model via AIC or likelihood ratio test to your base model, or simply check if the ZIP term in glmmTMB is significant.

```
m5 = glmmTMB(count ~ spp + mined + (1|site), family = nbinom2, ziformula = ~1, data = Salamanders)
summary(m5)
```

```
Family: nbinom2 ( log )
Formula:      count ~ spp + mined + (1 | site)
Zero inflation: ~1
Data: Salamanders
```

AIC	BIC	logLik	deviance	df.resid
1674.4	1723.5	-826.2	1652.4	633

Random effects:

Conditional model:

```
Groups Name      Variance Std.Dev.
site (Intercept) 0.2944   0.5426
Number of obs: 644, groups: site, 23
```

Dispersion parameter for nbinom2 family (): 0.942

Conditional model:

	Estimate	Std. Error	z value	Pr(> z)	
(Intercept)	-1.6832	0.2742	-6.140	8.28e-10	***
sppPR	-1.3197	0.2875	-4.591	4.42e-06	***
sppDM	0.3686	0.2235	1.649	0.099047	.
sppEC-A	-0.7098	0.2530	-2.806	0.005016	**
sppEC-L	0.5714	0.2191	2.608	0.009105	**
sppDES-L	0.7929	0.2166	3.660	0.000252	***
sppDF	0.3120	0.2329	1.340	0.180329	
minedno	2.2633	0.2838	7.975	1.53e-15	***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Zero-inflation model:

	Estimate	Std. Error	z value	Pr(> z)
(Intercept)	-16.41	4039.11	-0.004	0.997

In this case, we have no evidence for zero-inflation. To see an example where you can find zero-inflation, do the Owl case study below.

9.4 Interpreting interactions in GLMs

A significant problem with interpreting GLMs is the interpretation of slopes in the presence of other variables, in particular interactions. To understand this problem, let's first confirm to ourselves: if we simulate data under the model assumptions, parameters will be recovered as expected.

```
library(effects)
set.seed(123)
trt = as.factor(sample(c("ctrl", "trt"), 5000, replace= T))
concentration = runif(5000)

response = plogis(0 + 1 * (as.numeric(trt) - 1) + 1*concentration)
survival = rbinom(5000, 1, prob = response)

dat = data.frame(trt = trt,
                  concentration = concentration,
                  survival = survival)

m1 = glm(survival ~ trt * concentration, data = dat, family = "binomial")
summary(m1)
```

Call:

```
glm(formula = survival ~ trt * concentration, family = "binomial",
    data = dat)
```

Deviance Residuals:

	Min	1Q	Median	3Q	Max
	-2.0767	-1.2595	0.6357	0.8578	1.1457

Coefficients:

	Estimate	Std. Error	z value	Pr(> z)
(Intercept)	0.07491	0.08240	0.909	0.363
trttrt	0.93393	0.12731	7.336	2.20e-13 ***
concentration	0.89385	0.14728	6.069	1.29e-09 ***
trttrt:concentration	0.13378	0.23675	0.565	0.572

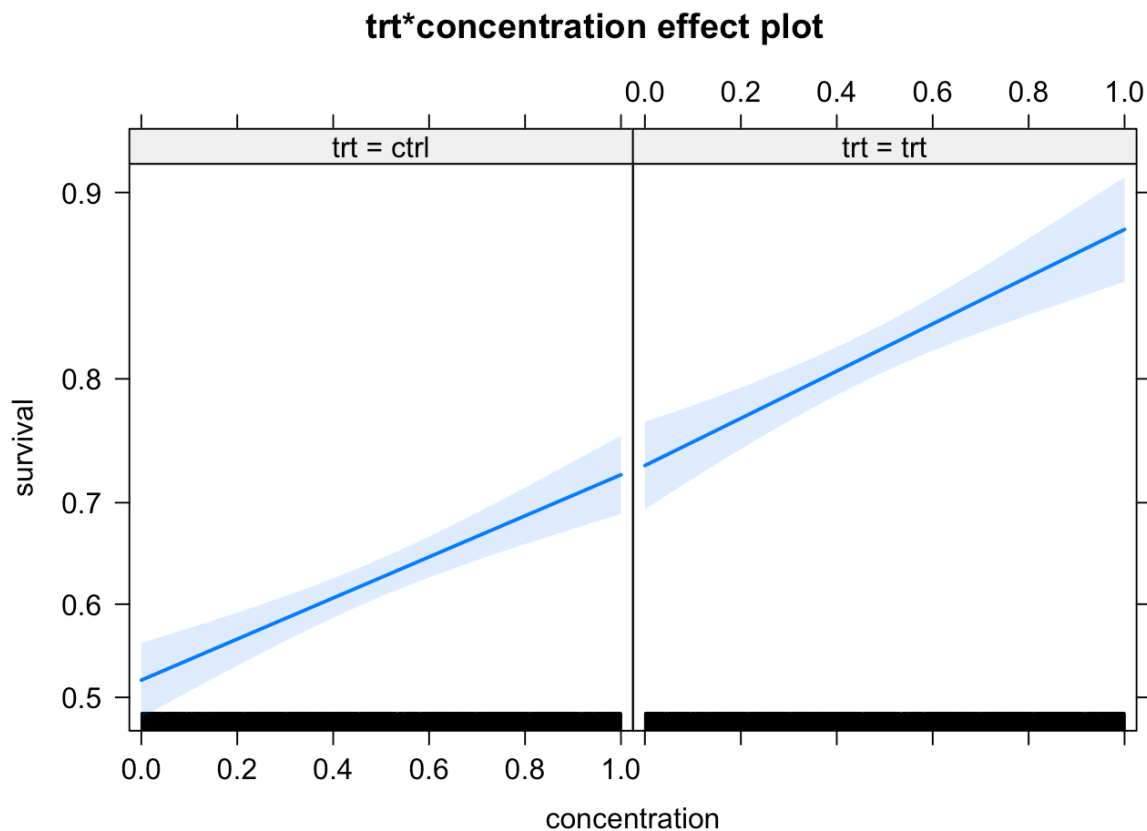
Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for binomial family taken to be 1)

Null deviance: 5920.1 on 4999 degrees of freedom
 Residual deviance: 5622.9 on 4996 degrees of freedom
 AIC: 5630.9

Number of Fisher Scoring iterations: 4

```
plot(allEffects(m1))
```



The problem with this, however, is the condition that we “simulate data under model assumptions”, which includes the nonlinear link function. Let’s have a look what happens if we simulate data differently: in this case, we just assume that treatment changes the overall probability of survival (from 45% to 90%), and the concentration increases the survival by up to 10% for each group. We may think that we don’t have an interaction in this case, but the model finds one

```

response = 0.45 * as.numeric(trt) + 0.1*concentration
survival = rbinom(5000, 1, response)

dat = data.frame(trt = trt,
                  concentration = concentration,
                  survival = survival)

m2 = glm(survival ~ trt * concentration,
          data = dat, family = "binomial")
summary(m2)

```

Call:

```
glm(formula = survival ~ trt * concentration, family = "binomial",
    data = dat)
```

Deviance Residuals:

Min	1Q	Median	3Q	Max
-2.8600	-1.0714	0.2821	1.0405	1.3101

Coefficients:

	Estimate	Std. Error	z value	Pr(> z)	
(Intercept)	-0.30719	0.08142	-3.773	0.000161	***
trttrt	2.46941	0.18030	13.696	< 2e-16	***
concentration	0.64074	0.14149	4.529	5.94e-06	***
trttrt:concentration	1.37625	0.39167	3.514	0.000442	***

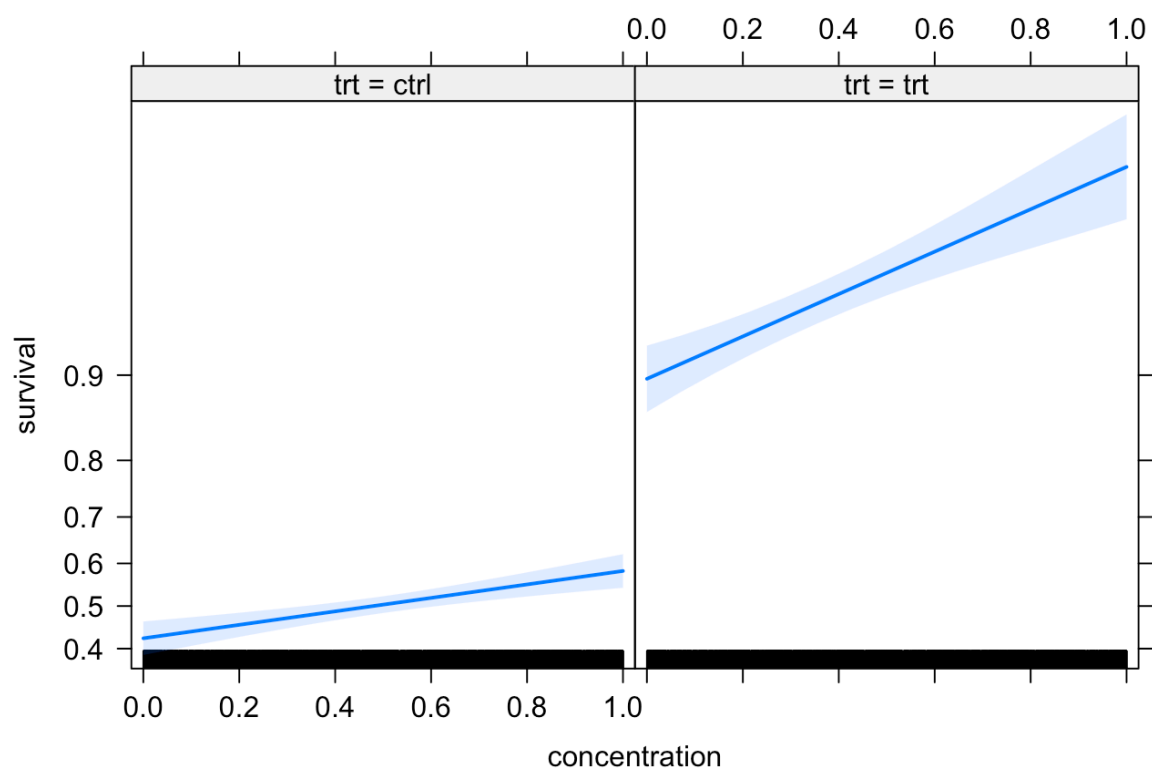
Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for binomial family taken to be 1)

Null deviance: 5848.4 on 4999 degrees of freedom
 Residual deviance: 4355.1 on 4996 degrees of freedom
 AIC: 4363.1

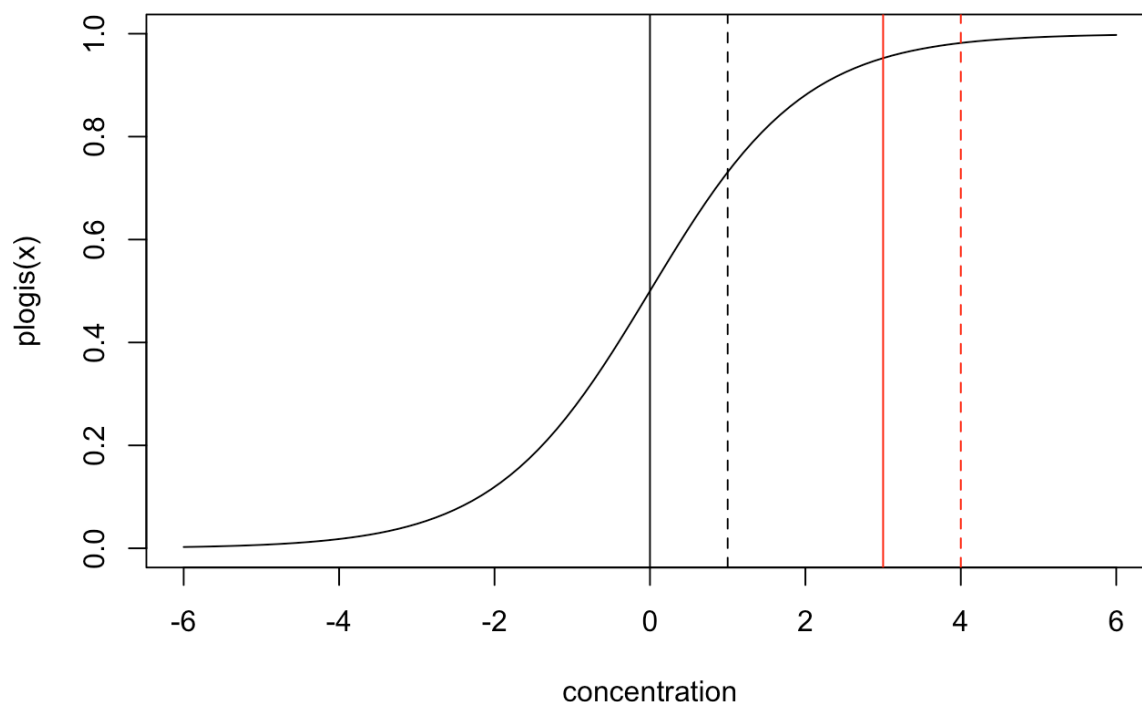
Number of Fisher Scoring iterations: 6

```
plot(allEffects(m2))
```

trt*concentration effect plot

It looks in the effect plots as if the slope is changing as well, but note that this because the effect plots scale the y axis according to the link - absolutely, the effect of concentration is 10% for both groups.

The reason is simple: if we plot the plogis function, it becomes obvious that at different base levels (which would be controlled by trt in our case), moving a unit in concentration has a different effect.



If we turn this around, this means that if we want the model to have the same effect of concentration at the response scale for both treatments, we must implement an interaction.

Whether this is a feature or a bug of GLMs depends a bit on the viewpoint. One could argue that, looking at survival, for example, it doesn't make sense that the concentration should have an effect of absolute 10% on top of the baseline created by trt for either 45% and 90% survival, and if we see such an effect, we should interpret this as an interaction, because relatively speaking, and increase of 45% to 55% is less important than an increase of 90% to 100%.

Still, this also means that main effects and interactions can change if you change the link function, and default links are not always natural from a biological viewpoint.

There are several ways to get better interpretable interactions. One option is that we could fit the last model with a binomial distribution, but with an identity link

```
m3 = glm(survival ~ trt * concentration,
          data = dat, family = binomial(link = "identity"))
```

Warning: step size truncated: out of bounds

Warning: step size truncated: out of bounds

Warning: step size truncated: out of bounds

Warning: step size truncated: out of bounds

Warning: step size truncated: out of bounds

Warning: step size truncated: out of bounds

Warning: step size truncated: out of bounds

Warning: step size truncated: out of bounds

Warning: step size truncated: out of bounds

Warning: step size truncated: out of bounds

Warning: step size truncated: out of bounds

Warning: glm.fit: algorithm stopped at boundary value

```
summary(m3)
```

Call:

```
glm(formula = survival ~ trt * concentration, family = binomial(link = "identity"),
     data = dat)
```

Deviance Residuals:

Min	1Q	Median	3Q	Max
-3.2649	-1.0716	0.3023	1.0398	1.3103

Coefficients:

	Estimate	Std. Error	z value	Pr(> z)
(Intercept)	0.42369	0.02010	21.079	< 2e-16 ***
trttrt	0.48388	0.02174	22.254	< 2e-16 ***
concentration	0.15935	0.03483	4.575	4.77e-06 ***
trttrt:concentration	-0.06690	0.03582	-1.868	0.0618 .

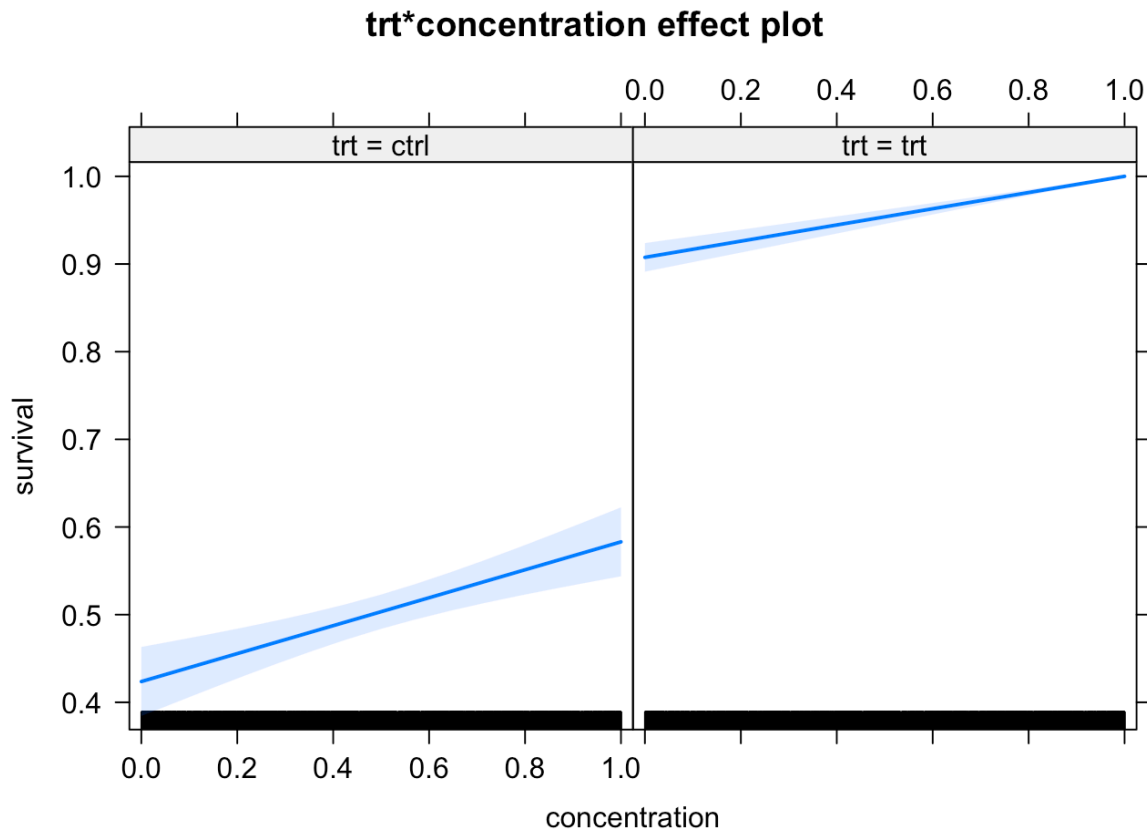
Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for binomial family taken to be 1)

Null deviance: 5848.4 on 4999 degrees of freedom
 Residual deviance: 4347.2 on 4996 degrees of freedom
 AIC: 4355.2

Number of Fisher Scoring iterations: 12

```
plot(allEffects(m3))
```



Now, we can interpret effects and interactions exactly like in a linear regression.

Another option is to look at the predicted effects at the response scale, e.g. via the effect plots, and interpret from there if we have an interaction according to what you would define as one biologically. One option to do this is the margins package.

Note

If effect directions change in sign, they will do so under any link function (as they are always monotonous), so changes in effect direction are robust to this problem.

9.5 New considerations for GLMMs

As we saw, in principle, random effects can be added to GLMs very much in the same way as before. Note, however, that there is a conceptual difference:

1. In an LMM, we had $y = f(x) + \text{RE} + \text{residual}$, where both RE and residual error were normal distributions
2. In a GLMM, we have $y = \text{dist} \leftarrow \text{link}^{-2} \leftarrow f(x) + \text{RE}$, so the normally distributed RE goes into another distribution via the link function

This has a number of consequences that may be unexpected if you don't think about it.

9.5.1 Unconditional vs. marginal predictions

One of those is that, if you have a nonlinear link function, predictions based on the fixed effects (unconditional) will not correspond to the mean of your data, or the average of the model predictions including the random effect (marginal predictions). To understand this, imagine we have a logistic regression, and our fixed effects predict a mean of 1. Then the unconditional prediction at the response scale

```
plogis(1)
```

```
[1] 0.7310586
```

Now, let's try to see what the average (marginal) prediction over all random effects is. We add a random effect with 20 groups, $sd = 1$. In this case, there will be additional variation around the random intercept.

```
mean(plogis(1 + rnorm(20, sd = 1)))
```

```
[1] 0.6966425
```

The value is considerably lower than the unconditional prediction, and this value is also what you would approximately get if you take the mean of your data.

Whether this is a problem or not is a matter of perspective. From statistical viewpoint, assuming that your model assumptions correspond to the data-generating model, there is nothing wrong with the unconditional (fixed effect) prediction not corresponding to the mean of the data. From a practical side, however, many people are unsatisfied with this, because they want to show predictions against data. Unfortunately (to my knowledge), there is no function to automatically create marginal predictions. A quick fix would be to create conditional predictions, add random effects on top as above (with the estimated variances) and push them through the corresponding link function.

9.6 Case Studies

9.6.1 Binomial GLM - Snails

The following dataset contains information about the occurrences of 3 freshwater snail species and their infection rate with schistosomiasis (freshwater snails are intermediate hosts). The data and the analysis is from [Rabone et al. 2019](#).

```
library(EcoData)
?snails
```

Help on topic 'snails' was found in the following packages:

Package	Library
EcoData	/Library/Frameworks/R.framework/Versions/4.2/Resources/library
MASS	/Library/Frameworks/R.framework/Versions/4.2/Resources/library

Using the first match ...

The dataset has data on three snail species: *Biomphalaria pfeifferi* (BP), *Bulinus forskalii* (BF), and *Bulinus truncatus* (BT). For this task, we will focus only on *Bulinus truncatus* (BT).

We want to answer two questions:

1. What are the variables that explain the occurrence of the *Bulinus truncatus* species (=Species distribution model)
2. What are the variables that influence the infection of the snails with Schistosomiasis

Tasks:

- Build two binomial GLMs (see hints) to explain the presence and the infection rate of *Bulinus truncatus* (BT)

- Add environmental variables and potential confounders (see ?snails for an overview of the variables and if you want, you can also take a look at the original paper)
- Check residuals with DHARMA, check for misfits (plot residuals against variables)
- Which variables explain the presence of BT and which have an effect on the infection rates?

Hints

Prepare dataset before the analysis:

```
data = EcoData::snails
data$sTemp_Water = scale(data$Temp_Water)
data$spH = scale(data$pH)
data$swater_speed_ms = scale(data$water_speed_ms)
data$swater_depth = scale(data$water_depth)
data$sCond = scale(data$Cond)
data$swmo_prec = scale(data$wmo_prec)
data$year = scale(data$year)
data$sLat = scale(data$Latitude)
data$sLon = scale(data$Longitude)
data$sTemp_Air = scale(data$Temp_Air)
# Remove NAs
rows = rownames(model.matrix(~sTemp_Water + spH + sLat + sLon + sCond + seas_wmo+ swmo_prec + swater_speed_ms + swater
data = data[rows, ]
```

1. Factors which drive the occurrence of BT

Minimal model:

```
model1 = glm(bt_pres~ sTemp_Water,
             data = data, family = binomial)
```

Add environmental variables such as Temperature, pH, water metrics (e.g. water speed etc). Which variables could be potential confounders (year? locality? Season?....)

2. Factors which affect the infection rate

We use a k/n binomial model:

```
model2 = glm(cbind(BT_pos_tot, BT_tot - BT_pos_tot) ~ sTemp_Water ,
             data = data[data$BT_tot > 0, ], family = binomial)
```

Add environmental variables such as Temperature, pH, water metrics (e.g. water speed etc). Which variables could be potential confounders (year? locality? Season?....)

Solution - Species distribution model

Solution - Infection rate

9.6.2 Elks

The data for this example is not in the ecodata package, but will be distributed in the course.

The data contains observations of elk presence / absence in various places in Canada. The ecological question is if roads disturb the elks.

Task: perform an analysis of the data to understand if roads have a causal effect on elk presence. Correct for appropriately for confounders! Check the residuals of your model to make sure it's properly specified!

9.6.3 Making spatial predictions with a logistic regression - Elephant SDM

To demonstrate how to fit a species distribution model and extrapolate it in space, let's have a look at the elephant dataset, which is contained in EcoData

```
library(EcoData)
?elephant
```

The object elephant contains two subdatasets

1. elephant\$occurrenceData contains presence / absence data as well as bioclim variables (environmental predictors) for the African elephant
2. elephant\$predictionData data with environmental predictors for spatial predictions

The environmental data consists of 19 environmental variables, called bio1 through bio19, which are public and globally available bioclimatic variables (see <https://www.worldclim.org/data/bioclim.html> for a description of the variables). For example, bio1 is the mean annual temperature. No understanding of these variables is required for the task, the only difficulty is that many of them are highly correlated because they encode similar information (e.g. there are several temperature variables).

The goal of this exercise is to fit a logistic regression model based on the observed presence / absences, and then make new predictions of habitat suitability in space across Africa based on the fitted model. Thus, our workflow consists of two steps:

1. building and optimizing the predictive model, and
2. using the predictive model to make predictions on new data and visualizing the results.

Here an example of how you could do this

Build predictive model:

```
model = glm(Presence~bio1, data = elephant$occurrenceData, family = binomial()) # minimum model
```

To check the predictive power of the model, you can either use

```
AIC(model)
```

```
[1] 3511.176
```

```
library(pROC)
auc(elephant$occurrenceData$Presence, predict(model, type = "response"))
```

Area under the curve: 0.7053

The AUC is a common measure of goodness of fit for binary classification. However, it doesn't penalize for complexity, as the AIC. Better to look at it under cross-validation

```
library(boot)
res = cv.glm(elephant$occurenceData, model, cost = auc, K=5)
```

```
res$delta
```

```
[1] 0.706149 0.706149
```

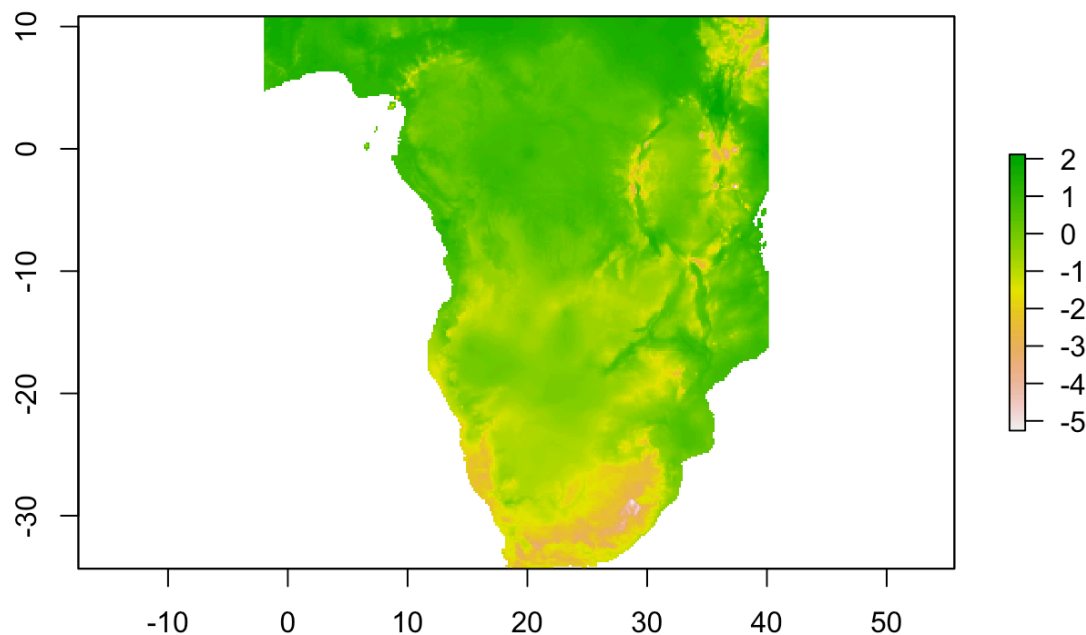
Currently, fit and validation AUC are nearly identical, because we have a very simple model. However, for more complex models, this could change.

- Drop some of the highly correlated variables (don't use all of them).
- Use quadratic effects
- Use `stepAIC` or the `dredge` function to optimize the predictive model

Make new predictions

The data for making spatial predictions is in `elephant$predictionData`. This new dataset is not a `data.frame` but a raster object, which is a special data class for spatial data. You can plot one of the predictors in the following way.

```
library(sp)
library(raster)
plot(elephant$predictionData$bio1)
```



As our `new_data` object is not a typical `data.frame`, we are not using the standard `predict` function for a `glm`, which is `predict.glm`, but the `predict` function from the raster object (which internally transforms the `new_data` into a classical

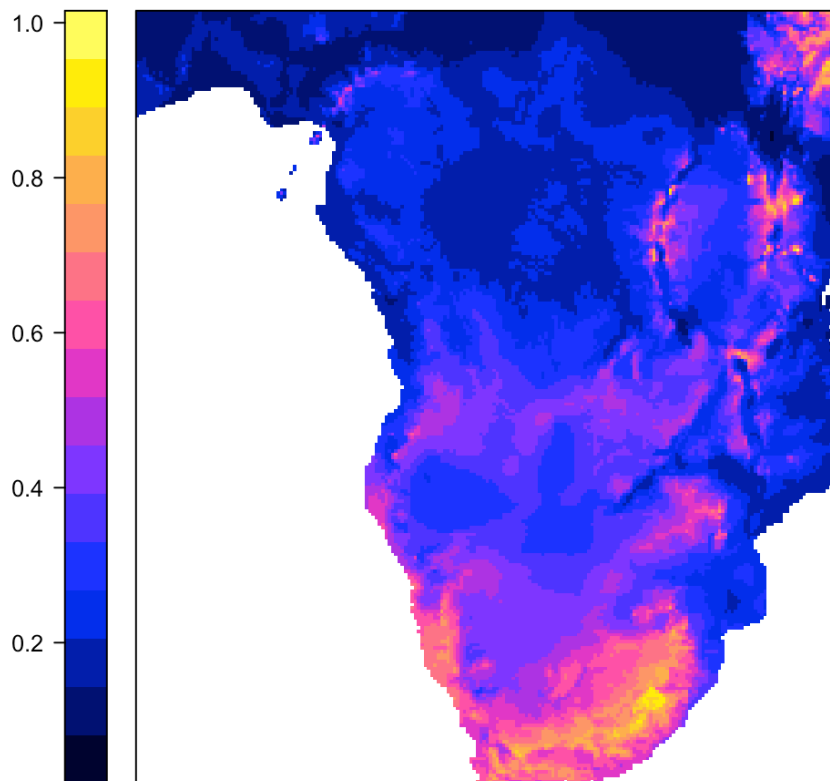
data.frame, pass then the data.frame to our model, and then transforms the output back to a raster object). Therefore, the syntax is slightly different to how we previously used predict().

```
predictions = predict(elephant$predictionData, model = model, type = "response")
head(as.data.frame(predictions))
```

```
      layer
1 0.1195466
2 0.1148339
3 0.1102836
4 0.1080683
5 0.1037547
6 0.1016556
```

The advantage of the raster object is that we can directly use it to create a map (the raster object has coordinates for each observation):

```
spplot(predictions, colorkey = list(space = "left") )
```



Task: play around with the logistic regression to improve predictive accuracy. You can check predictive accuracy by looking at AIC or by taking out a bit of the data for validation, e.g. by AUC. When improving the predictive power of the model, does the map change?